

Aporte individual: Modelo conceptual de la red social estudiantil pascualina

Nombre: Samuel Hincapie Perez

Tecnología en Desarrollo de Software

Base de Datos 1

Profesora: Oralia Cortes Grajales

Requerimientos funcionales que sustentan el modelo

RF1. Registro de estudiantes

El sistema debe permitir registrar estudiantes con sus datos básicos, como código, nombre, correo institucional, estado y fecha de registro.

RF2. Asociación con un programa académico

Cada estudiante debe estar relacionado con un programa académico, mientras que un programa puede tener varios estudiantes registrados.

RF3. Registro de habilidades

El sistema debe permitir que un estudiante tenga varias habilidades y que una misma habilidad pueda ser compartida por diferentes estudiantes.

RF4. Registro de intereses

El sistema debe permitir que los estudiantes tengan varios intereses y que un mismo interés pueda estar relacionado con diferentes estudiantes.

RF5. Seguimiento entre estudiantes

El sistema debe permitir que los estudiantes sigan a otros estudiantes y que también puedan ser seguidos por varias personas.

RF6. Publicación de contenido

El sistema debe permitir que los estudiantes publiquen contenido. Cada publicación debe estar relacionada con el estudiante que la realizó.

RF7. Creación de grupos

El sistema debe permitir que un estudiante cree un grupo, teniendo cada grupo un único creador.

RF8. Unión a grupos

El sistema debe permitir que los estudiantes se unan a diferentes grupos, sin importar quién haya creado cada grupo.

RF9. Creación de eventos

El sistema debe permitir que un estudiante cree eventos, teniendo cada evento un único creador.

RF10. Participación en eventos

El sistema debe permitir que los estudiantes se inscriban o participen en diferentes eventos.

RF11. Organización de eventos

El sistema debe permitir que un grupo organice eventos, de manera que un grupo pueda organizar varios eventos.

Análisis de necesidades por entidad

Estudiante

La entidad **Estudiante** es la principal del modelo, porque prácticamente todas las actividades del sistema están relacionadas con ella. Aquí se guardan los datos básicos de cada estudiante, como su nombre, correo institucional, estado y fecha de registro.

También se relaciona con programas, habilidades, intereses, publicaciones, grupos, eventos y otros estudiantes. Por eso se puede decir que es el centro del modelo, ya que las demás entidades representan actividades o información relacionada con los estudiantes.

Programa

La entidad **Programa** se utiliza para almacenar la información de los programas académicos, como su código, nombre y facultad.

Se decidió manejarla como una tabla independiente para no repetir estos datos en cada estudiante. Por ejemplo, si varios estudiantes pertenecen al mismo programa, no sería necesario escribir nuevamente el nombre del programa y la facultad en cada registro.

Esto ayuda a evitar la repetición de información y facilita realizar cambios cuando sea necesario.

Habilidad

La entidad **Habilidad** representa las diferentes capacidades que pueden tener los estudiantes, como programación, liderazgo, diseño, entre otras.

Se encuentra separada de Estudiante porque una misma habilidad puede pertenecer a muchos estudiantes. Por ejemplo, varios estudiantes pueden tener la habilidad de programación.

Si las habilidades se guardaran directamente en Estudiante, sería más complicado consultar cuáles estudiantes tienen una determinada habilidad.

Interes

La entidad **Interes** representa los temas o actividades que les gustan a los estudiantes, como música, videojuegos, fotografía o tecnología.

Se maneja de forma independiente porque un mismo interés puede estar relacionado con varios estudiantes. Además, se separa de Habilidad porque no es lo mismo tener conocimiento o capacidad en algo que simplemente tener interés en ese tema.

Publicacion

La entidad **Publicacion** representa el contenido que los estudiantes comparten en la plataforma.

Se creó como una tabla independiente porque un estudiante puede realizar muchas publicaciones y cada una necesita guardar información propia, como el tipo de publicación, el contenido y la fecha en que fue realizada.

También tiene una clave foránea que permite saber qué estudiante hizo cada publicación.

Grupo

La entidad **Grupo** representa comunidades o espacios de integración dentro de la plataforma. Por ejemplo, podría ser un grupo de estudio, un club o un semillero.

La tabla guarda información como el nombre, tipo y fecha de creación del grupo. Además, tiene el campo `codigo_estudiante`, que permite identificar al estudiante que creó ese grupo.

Aquí es importante diferenciar entre el estudiante que crea el grupo y los estudiantes que se unen a él, porque no necesariamente son la misma persona.

Evento

La entidad **Evento** representa actividades que se realizan dentro de la plataforma, como charlas, talleres, reuniones o actividades académicas.

Tiene información como nombre, hora y lugar. Además, en el modelo aparecen dos claves foráneas relacionadas con estudiantes y grupos:

codigo_estudiante: identifica al estudiante que creó el evento.

codigo_grupo: identifica al grupo que organiza el evento.

Esto permite diferenciar quién creó el evento y qué grupo está encargado de organizarlo.

Análisis de relaciones: cardinalidad y justificación

Para decidir cómo representar cada relación, se analizó cuántos registros de una entidad pueden relacionarse con la otra.

La regla utilizada fue la siguiente:

Si ambos lados pueden tener varios registros relacionados, se utiliza una tabla puente.

Si un lado tiene un único registro relacionado y el otro puede tener varios, se utiliza una clave foránea directa.

Relaciones muchos a muchos

Relación posee: Estudiante - Habilidad

La relación **posee** es de muchos a muchos porque un estudiante puede tener varias habilidades y una misma habilidad puede pertenecer a varios estudiantes.

Por ejemplo, un estudiante puede tener conocimientos en programación, diseño y bases de datos, mientras que otros estudiantes también pueden tener esas mismas habilidades.

Por esta razón se creó la tabla puente **Estudiante_Habilidad**, que contiene:

codigo_estudiante

codigo_habilidad

Esta tabla permite registrar qué habilidades tiene cada estudiante sin repetir información.

Relación tiene: Estudiante - Interes

La relación **tiene** también es muchos a muchos.

Un estudiante puede tener varios intereses y un mismo interés puede estar asociado a diferentes estudiantes.

Por ejemplo, varios estudiantes pueden tener interés en los videojuegos, mientras que uno de ellos también puede tener interés en música y tecnología.

Para resolver esta relación se utiliza la tabla puente **Estudiante_Interes**, que contiene:

codigo_estudiante

codigo_interes

De esta forma se pueden registrar todas las combinaciones entre estudiantes e intereses.

Relación sigue: Estudiante - Estudiante

La relación **sigue** es un caso especial porque Estudiante se relaciona consigo misma.

Esto significa que un estudiante puede seguir a muchos otros y también puede ser seguido por muchos estudiantes.

Por ejemplo:

El estudiante 101 sigue al estudiante 205.

Para representar esta relación se creó la tabla **Seguimiento**, que contiene:

codigo_estudiante_seguidor

codigo_estudiante_seguido

fecha_seguimiento

Aunque ambos códigos vienen de la tabla Estudiante, cumplen funciones diferentes:

El seguidor es quien realiza el seguimiento.

El seguido es la persona que recibe ese seguimiento.

Por eso se necesitan dos claves foráneas hacia la misma tabla.

Relación se une a: Estudiante - Grupo

La relación **se une a** es muchos a muchos porque un estudiante puede pertenecer a varios grupos y un grupo puede tener muchos estudiantes.

Por ejemplo, un estudiante puede estar en un grupo de programación y también en uno de investigación.

Para resolver esta relación se utiliza la tabla **Estudiante_Grupo**, que contiene:

codigo_estudiante

codigo_grupo

fecha_union

La fecha de unión es importante porque permite saber cuándo un estudiante ingresó a un grupo específico.

Relación se une a: Estudiante - Evento

La relación entre Estudiante y Evento también es muchos a muchos, ya que un estudiante puede participar en varios eventos y un evento puede tener varios participantes.

Para resolverla se utiliza la tabla **Estudiante_Evento**, que contiene:

codigo_estudiante

codigo_evento

fecha_inscripcion

Esta tabla permite registrar qué estudiantes participan en cada evento y cuándo se inscribieron.

En estos casos se utilizan tablas puente porque no sería suficiente colocar una sola clave foránea en una de las tablas principales. Eso limitaría la cantidad de relaciones que se pueden registrar y se perdería información.

Relaciones uno a muchos

Relación pertenece a: Programa - Estudiante

Esta relación es uno a muchos porque un programa puede tener varios estudiantes, pero cada estudiante pertenece a un programa académico.

Por eso la clave foránea `codigo_programa` se encuentra en la tabla **Estudiante**.

No se necesita una tabla puente porque cada estudiante solo está asociado a un programa dentro del modelo.

Relación publica: Estudiante - Publicacion

Esta relación es uno a muchos porque un estudiante puede realizar varias publicaciones, pero cada publicación pertenece a un solo estudiante.

Por eso la tabla **Publicacion** contiene la clave foránea `codigo_estudiante`, que permite identificar al autor de cada publicación.

Relación crea: Estudiante - Grupo

La relación **crea** es uno a muchos porque un estudiante puede crear varios grupos, pero cada grupo tiene un único creador.

En el diagrama, esta relación se representa mediante el campo:

`codigo_estudiante` en la tabla **Grupo**.

Este campo permite saber qué estudiante creó cada grupo.

Es importante no confundir esta relación con **se une a**, porque crear un grupo y pertenecer a él son cosas diferentes.

Un estudiante puede crear un grupo y luego otros estudiantes pueden unirse mediante la tabla **Estudiante_Grupo**.

Relación crea: Estudiante - Evento

La relación **crea** entre Estudiante y Evento también es uno a muchos.

Un estudiante puede crear varios eventos, pero cada evento tiene un único creador.

Por eso la tabla **Evento** contiene el campo:

`codigo_estudiante`

Este campo identifica al estudiante que creó el evento.

Sin embargo, esto no significa que ese estudiante sea el único que participe en el evento, ya que los demás participantes se registran en la tabla

Estudiante_Evento.

Relación organiza: Grupo - Evento

La relación **organiza** es uno a muchos porque un grupo puede organizar varios eventos, mientras que cada evento está asociado a un grupo organizador.

Por eso la tabla **Evento** contiene la clave foránea:

codigo_grupo

Esta permite identificar qué grupo organiza cada evento.

Esta relación es diferente a la de crear, porque un estudiante puede crear el evento, pero un grupo puede ser el encargado de organizarlo.

Diferencia entre las relaciones parecidas

En el modelo hay algunas relaciones que pueden confundirse, por eso es importante diferenciarlas.

Crear un grupo

Indica quién fue el estudiante que creó el grupo.

Se representa con:

Grupo → codigo_estudiante

Unirse a un grupo

Indica qué estudiantes pertenecen a ese grupo.

Se representa con:

Estudiante_Grupo

Crear un evento

Indica qué estudiante creó el evento.

Se representa con:

Evento → codigo_estudiante

Participar en un evento

Indica qué estudiantes están inscritos o participan en el evento.

Se representa con:

Estudiante_Evento

Organizar un evento

Indica qué grupo está encargado de organizarlo.

Se representa con:

Evento → código_grupo

Aunque algunas relaciones parecen similares, cada una representa una acción diferente dentro del sistema.

Conclusión

En conclusión, mi modelo se diseñó teniendo en cuenta las necesidades principales del sistema y la forma en que se relacionan sus entidades.

Las relaciones muchos a muchos se resolvieron mediante tablas puente, como:

Estudiante_Habilidad.

Estudiante_Interes.

Seguimiento.

Estudiante_grupo.

Estudiante_Evento.

Por otro lado, las relaciones uno a muchos se resolvieron utilizando claves foráneas directas, como:

Estudiante con Programa.

Publicacion con Estudiante.

Grupo con Estudiante.

Evento con Estudiante.

Evento con Grupo.

De esta manera, el modelo permite representar correctamente la información de los estudiantes, sus habilidades, intereses, publicaciones, grupos y eventos, evitando repetición de datos.

